Student Performance Analysis and Deep Learning Classification

COURSE REPORT

Submitted by

Preethi S

Roll Number: 23AM048

Department: IV CSE (AIML)

Course: Exploratory Data Analysis and Visualisation

Submission Date: 15 October 2026

Abstract

This report presents an Exploratory Data Analysis (EDA) of the UCI Student Performance dataset, followed by a binary classification experiment using a Multi-Layer Perceptron (MLP). The mathematics-subject dataset contains 395 student records and 33 columns describing demographic, family, school, lifestyle, attendance, and grade-related information. EDA includes descriptive statistics, grade distribution, study-time distribution, absence analysis, grade comparisons, correlation analysis, and outlier detection. The modelling task labels students with a final grade (G3) of 10 or above as higher performance and students with a grade below 10 as lower performance. The previous-period grades G1 and G2, as well as target grade G3, are excluded from the input features to reduce direct target leakage. The MLP achieved 66.67% test accuracy and a ROC-AUC of 0.6263. The results show that the model identifies many higher-performing students but has difficulty identifying the lower-performance class. These findings should be interpreted as an educational data-analysis exercise rather than a reliable real-world student-risk system.

1. Introduction

Exploratory Data Analysis is used to inspect a dataset, understand the distribution of variables, identify data-quality issues, discover relationships, and guide subsequent modelling. Student performance is influenced by multiple interacting factors, so analysing demographic, family, study, attendance, and lifestyle variables can help reveal patterns in the available data.

The purpose of this project is to conduct a structured EDA and demonstrate a deep-learning classification workflow. The analysis begins with dataset inspection and visualization, continues through preprocessing and feature encoding, and ends with model evaluation, feature-importance analysis, and a small learning-rate comparison.

1.1 Objectives

- Load and inspect the Student Performance dataset and summarize its structure.
- Calculate descriptive statistics and create meaningful EDA visualizations.
- Check missing values, duplicate records, and potential outliers.
- Prepare numerical and categorical features for machine learning.
- Train an MLP to classify final-grade outcomes into lower and higher performance.
- Evaluate the model with accuracy, precision, recall, F1-score, ROC-AUC, and a confusion matrix.
- Examine permutation importance and compare selected learning rates.

2. Dataset Description

The notebook loads the mathematics-subject file, student-mat.csv, from the UCI Student Performance dataset. The dataset has 395 rows and 33 columns: 16 numeric columns and 17 categorical/text columns. The target source variable is G3, the final grade, with values ranging from 0 to 20.

Dataset property	Observed value
Dataset	UCI Student Performance – mathematics subset
Records	395 students
Columns	33
Numeric columns	16
Categorical/text columns	17
Final grade variable	G3 (0–20)
Mean G3	10.42
Median G3	11
Mean age	16.70 years
Mean absences	5.71

Examples of variables include age, sex, address, parents' education and jobs, studytime, failures, school support, family support, absences, and grades G1, G2, and G3. For the classification experiment, G3 is converted to a binary label: 0 means lower performance (G3 < 10) and 1 means higher performance (G3 ≥ 10).

3. Data Understanding and Preprocessing

3.1 Initial inspection

The notebook checks column names, data types, a preview of records, descriptive summaries, missing-value counts, and duplicate rows. All 395 records were present across the 33 columns. The initial check reported zero missing values and zero duplicate rows.

3.2 Missing values and duplicates

The cleaning stage removes duplicate rows if any are found and fills missing numerical values with the median and categorical values with the mode. In this execution, the dataset contained no missing values or duplicate rows, so the cleaning steps did not change the record count.

3.3 Outlier detection

Potential outliers in the absences column were identified using the interquartile range (IQR) rule. Q1 was 0 and Q3 was 8, giving IQR = 8. The lower bound was −12 and the upper bound was 20. Fifteen records were flagged as potential high-absence outliers. These observations were retained because

unusually high absence counts can be genuine student records; an outlier flag alone does not justify deletion.

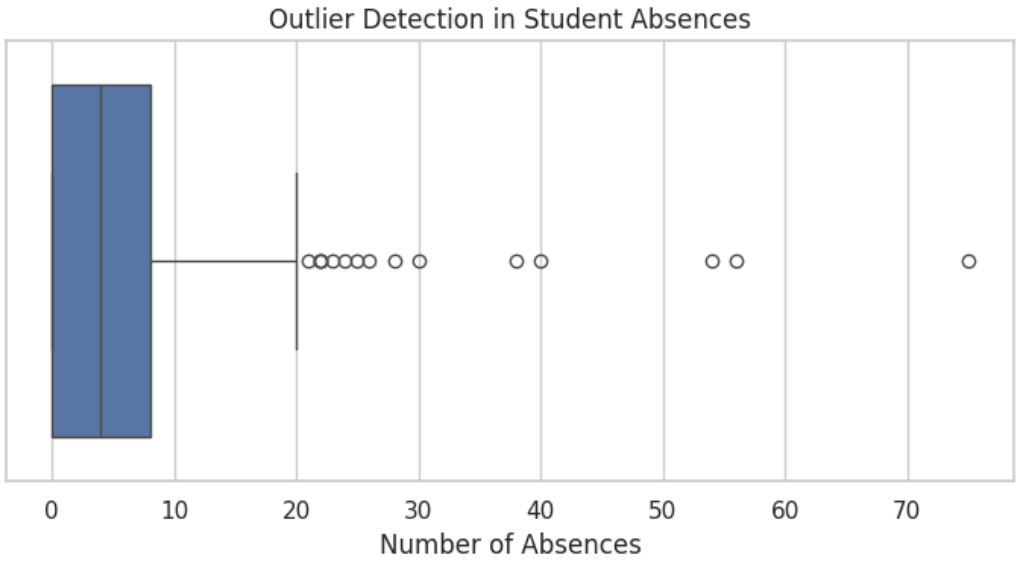


Figure 1. Box plot used to identify potential outliers in student absences.

3.4 Feature preparation and data split

The target label is created from G3 using the threshold of 10. G1, G2, and G3 are removed from X before training; this avoids using the final grade itself and its two earlier grade columns as model inputs. The dataset is split using stratified sampling: 70% for training, 15% for validation, and 15% for testing.

Subset	Number of records	Purpose
Training	276	Fit model parameters
Validation	59	Monitor training and compare hyperparameters
Testing	60	Final evaluation on held-out data

A ColumnTransformer applies median imputation and StandardScaler to numerical features, and most-frequent imputation plus one-hot encoding to categorical features. The fitted preprocessing pipeline is trained on the training set only, then applied to the validation and test sets. The transformed input has 56 features.

4. Exploratory Data Analysis

4.1 Final-grade distribution

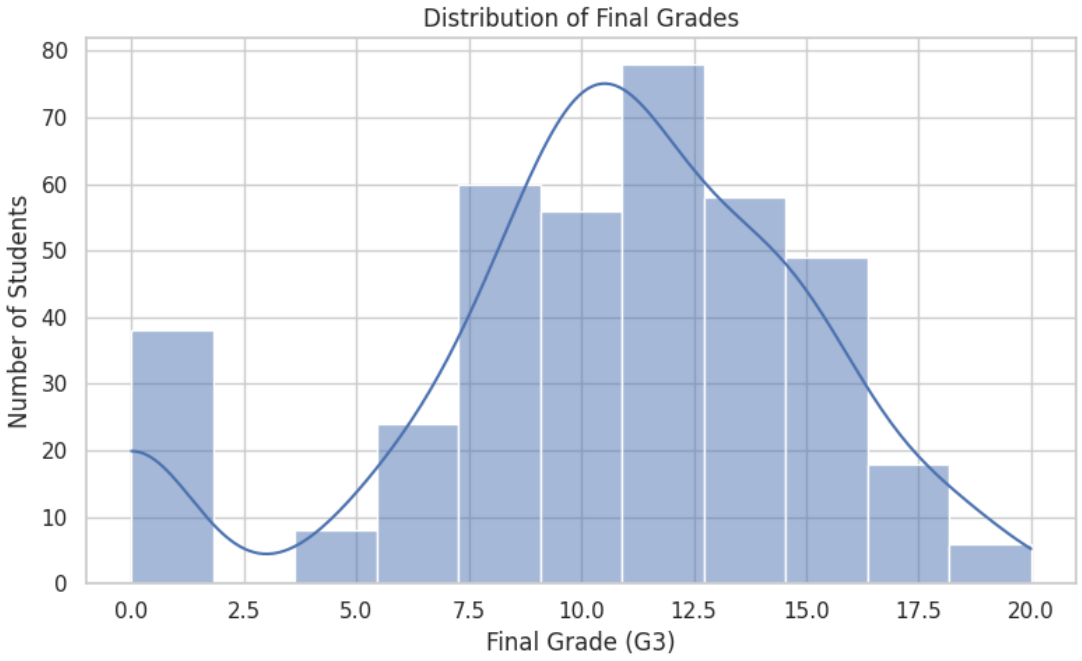


Figure 2. Distribution of final grades (G3).

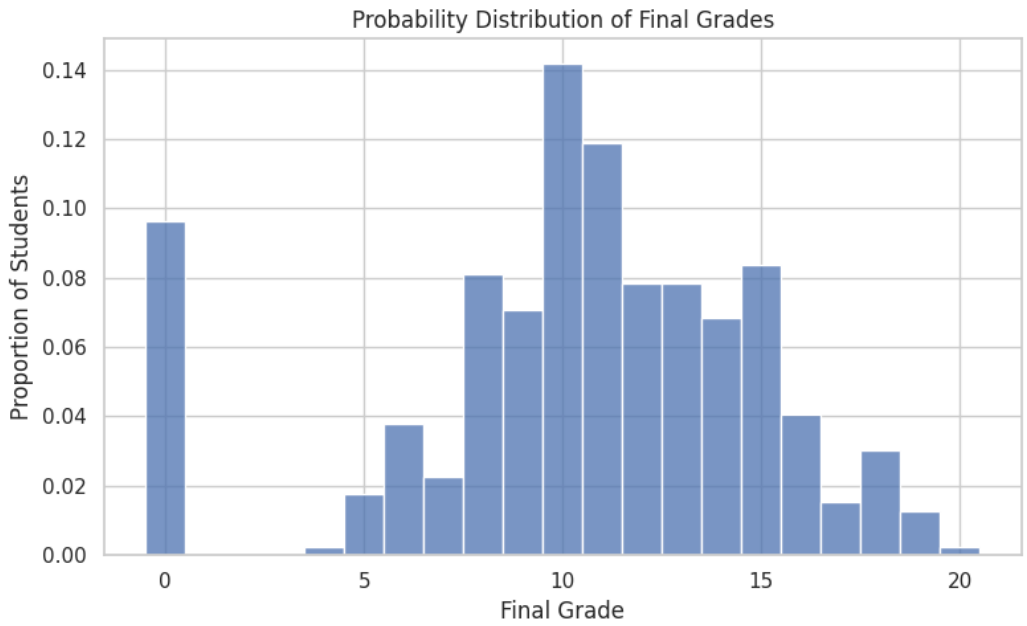


Figure 3. Probability distribution of each observed final-grade value.

The mean final grade is 10.42 and the median is 11. The distribution includes students with a final grade of zero as well as students reaching the maximum grade of 20. In the notebook's probability table, the most common individual grade is 10 (approximately 14.18%), followed by 11

(approximately 11.90%). This indicates that many students are clustered around the pass threshold used for the binary target.

4.2 Weekly study-time categories

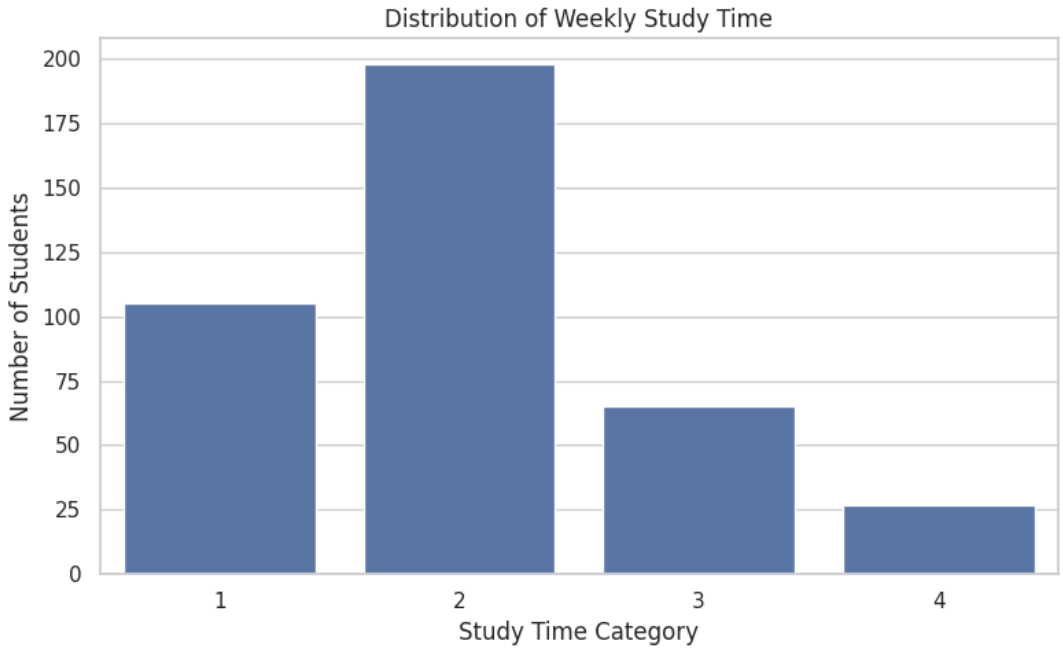


Figure 4. Count of students in each study-time category.

The studytime variable is ordinal and represents weekly study-time categories. The count plot helps show how the records are distributed across the four coded categories. A count plot describes the sample composition; it does not by itself prove that more study time causes higher grades.

4.3 Absences and final grades

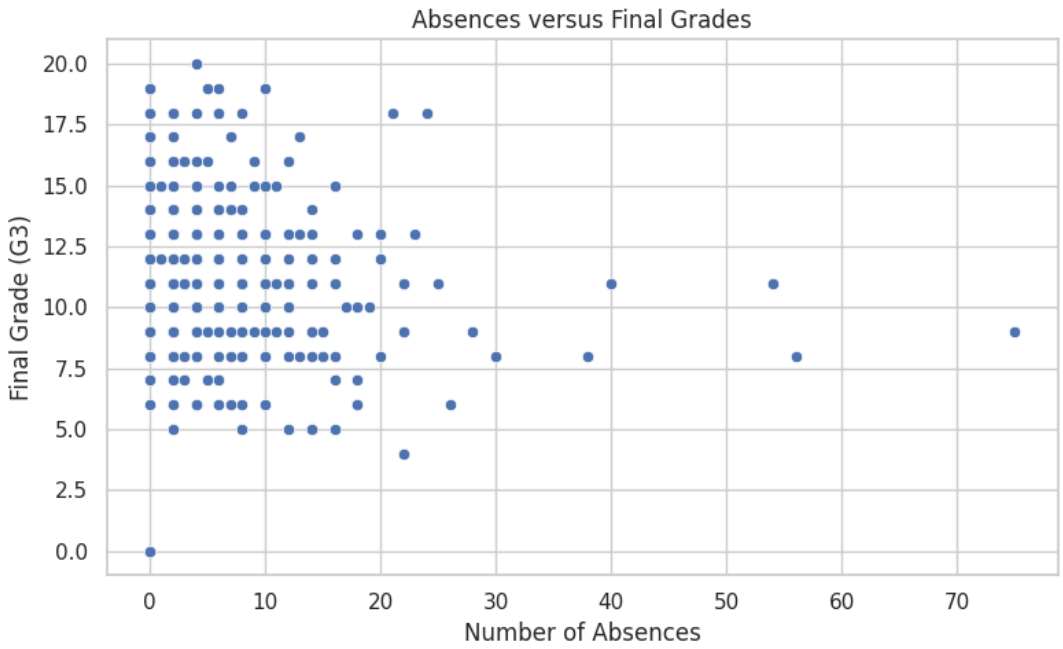


Figure 5. Scatter plot of absences against final grade.

The scatter plot compares absence counts with final grades. Absence values vary substantially, with most observations at lower counts and a small number at much higher counts. The points are spread across several grade levels, so attendance alone does not fully explain final performance.

4.4 Study time and final grades

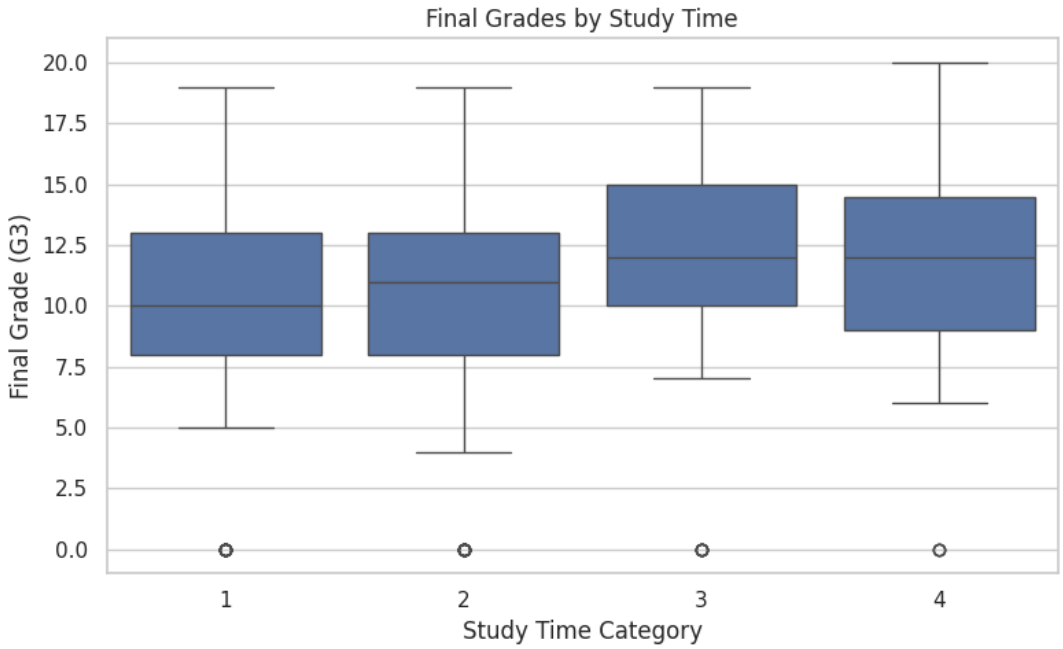


Figure 6. Final-grade distribution within each study-time category.

The box plot compares grade medians and spread across study-time groups. It is useful for identifying whether grade distributions differ across categories, while recognizing that groups may overlap and other factors may also contribute to performance.

4.5 Correlation among numerical variables

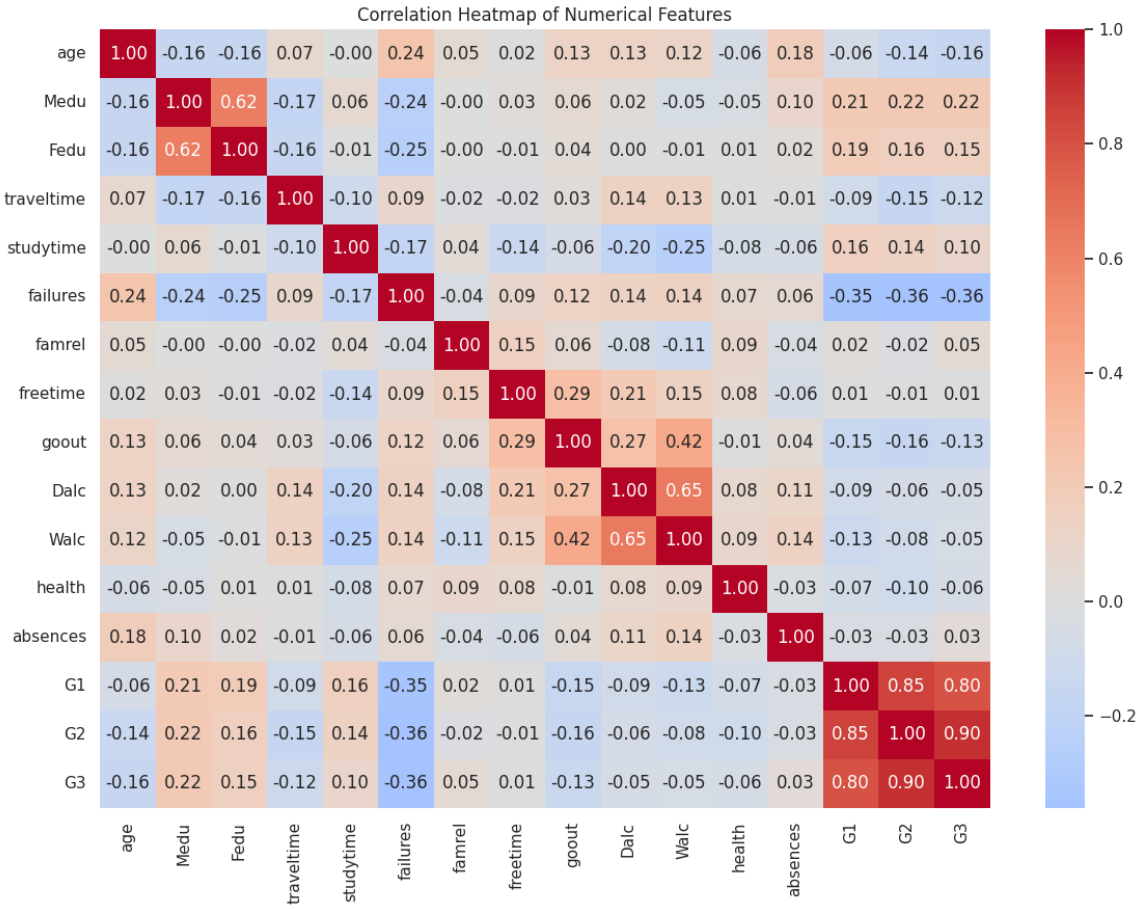


Figure 7. Correlation heatmap for numerical features.

The heatmap summarizes pairwise linear correlations among numeric columns. The grade columns G1, G2, and G3 are included in this EDA visualization, where they can be compared descriptively. They are excluded from the classification inputs. Correlation indicates association, not causation, and can miss non-linear relationships.

4.6 Descriptive statistics and distribution shape

The notebook calculates mean, median, standard deviation, minimum, maximum, and skewness for numerical variables. Average studytime is 2.04 on its coded scale, while average failures is 0.33. Absences have a mean of 5.71, median of 4, standard deviation of 8.00, and maximum of 75. The positive skewness of absences (3.67) indicates a right-skewed distribution, with a small number of students reporting substantially more absences than the majority.

5. Deep Learning Model

5.1 Target definition

The task is binary classification rather than direct prediction of the 0–20 grade. A student is labelled higher performance if G3 ≥ 10; otherwise the label is lower performance. There are 265 higher-performance records and 130 lower-performance records in the full dataset, so the classes are imbalanced.

5.2 MLP architecture

Layer	Configuration	Trainable parameters
Input	56 preprocessed features	—
Dense 1	64 neurons, ReLU	3,648
Dropout 1	Rate 0.20	0
Dense 2	32 neurons, ReLU	2,080
Dropout 2	Rate 0.10	0
Output	1 neuron, sigmoid	33
Total	5,761 trainable parameters	5,761

The network is trained with the Adam optimizer, learning rate 0.001, binary cross-entropy loss, and batch size 32. Training is configured for up to 100 epochs, with early stopping based on validation loss (patience 10) and restoration of the best weights. A fixed random seed of 42 is used for reproducibility.

6. Model Results and Evaluation

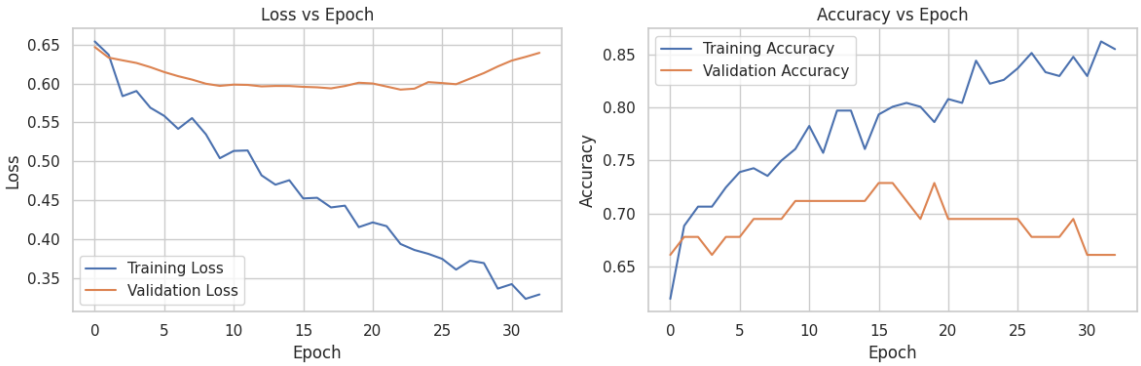


Figure 8. Training and validation loss and accuracy across epochs.

The learning curves show how the loss and accuracy change during training. Validation curves are important because a model that performs well only on training records may not generalize. The small dataset and the modest validation performance suggest that the model's predictive ability should be interpreted cautiously.

Metric	Test result
Accuracy	0.6667 (66.67%)

Precision – higher-performance class	0.7174
Recall – higher-performance class	0.8250
F1-score – higher-performance class	0.7674
ROC-AUC	0.6262

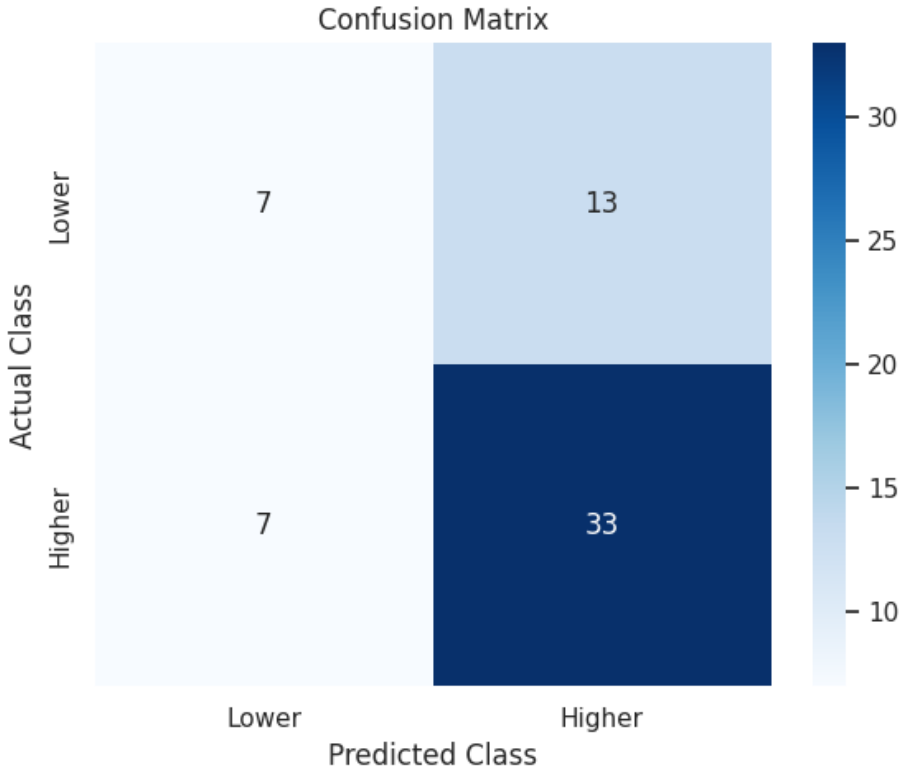


Figure 9. Confusion matrix for the 60-record test set.

The confusion matrix corresponds to 7 correctly identified lower-performance students, 13 lower-performance students predicted as higher performance, 7 higher-performance students predicted as lower performance, and 33 correctly identified higher-performance students. Thus, the model is better at recognizing higher performance (recall 0.825) than lower performance (recall 0.35).

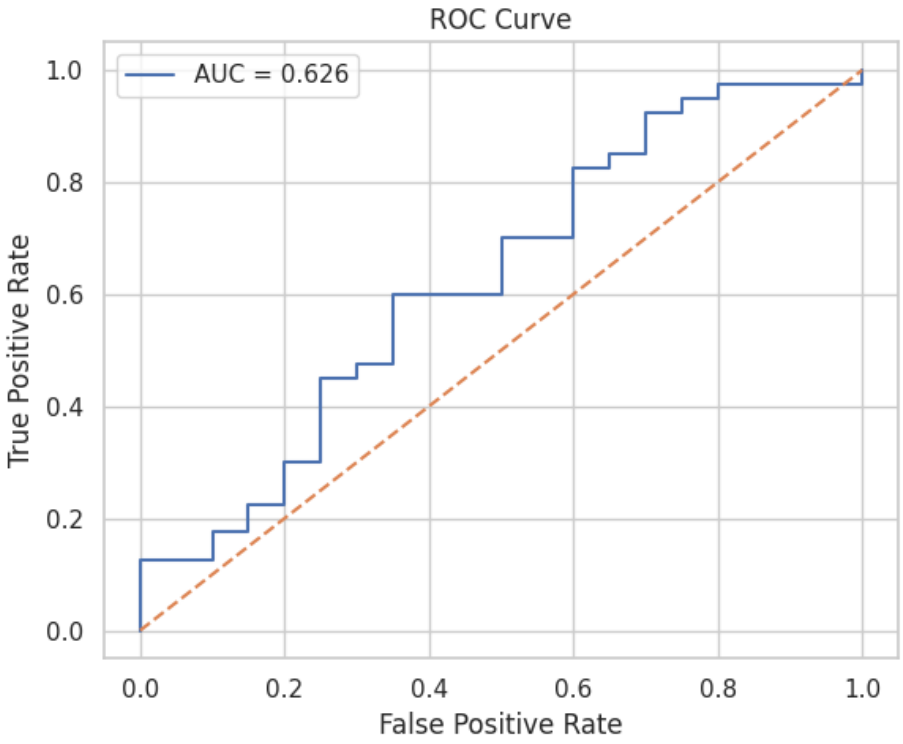


Figure 10. ROC curve and area under the curve for the test set.

The test ROC-AUC is 0.6262. This is above the 0.5 reference line but indicates limited class-separation ability. Accuracy alone would hide the weaker recall for lower-performing students, so the confusion matrix and class-wise precision/recall are important.

7. Feature Importance and Hyperparameter Comparison

7.1 Permutation feature importance

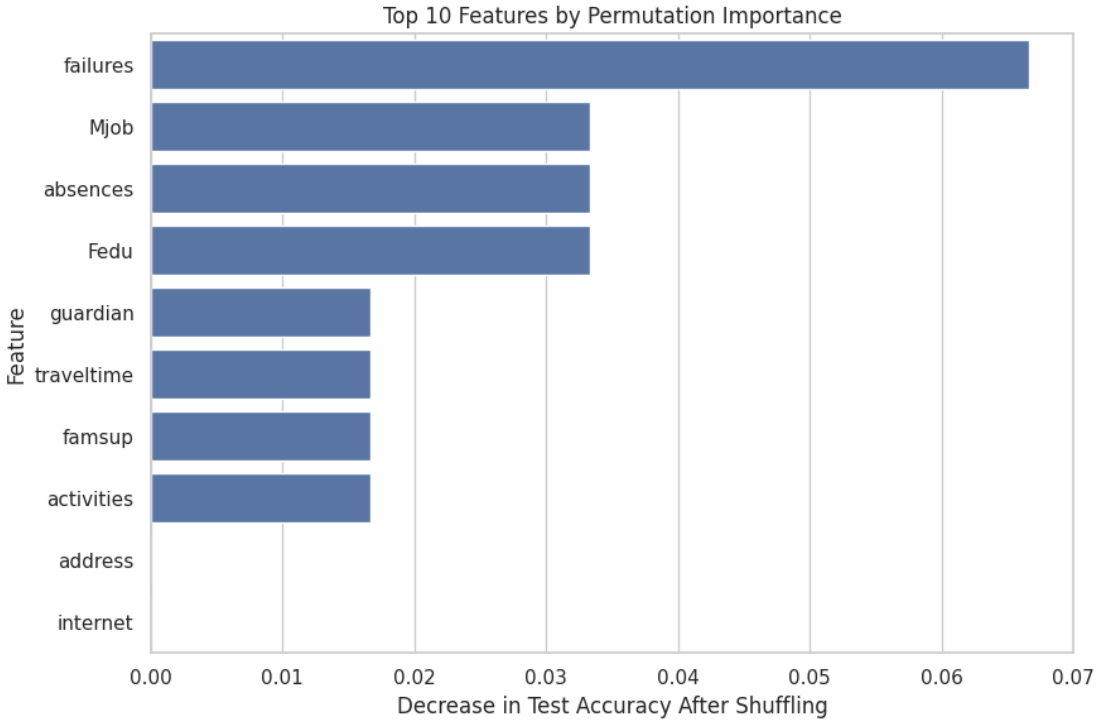


Figure 11. Ten features with the largest measured permutation-importance scores.

Permutation importance is estimated by shuffling one feature at a time and measuring the drop in test accuracy. In this run, failures has the largest score (0.0667), followed by Mjob, absences, and Fedu (each 0.0333). Guardian, traveltime, famsup, and activities each have a score of 0.0167. Address and internet have zero measured importance in the displayed top-ten list. These are model-specific associations, not proof of causal influence.

7.2 Learning-rate comparison

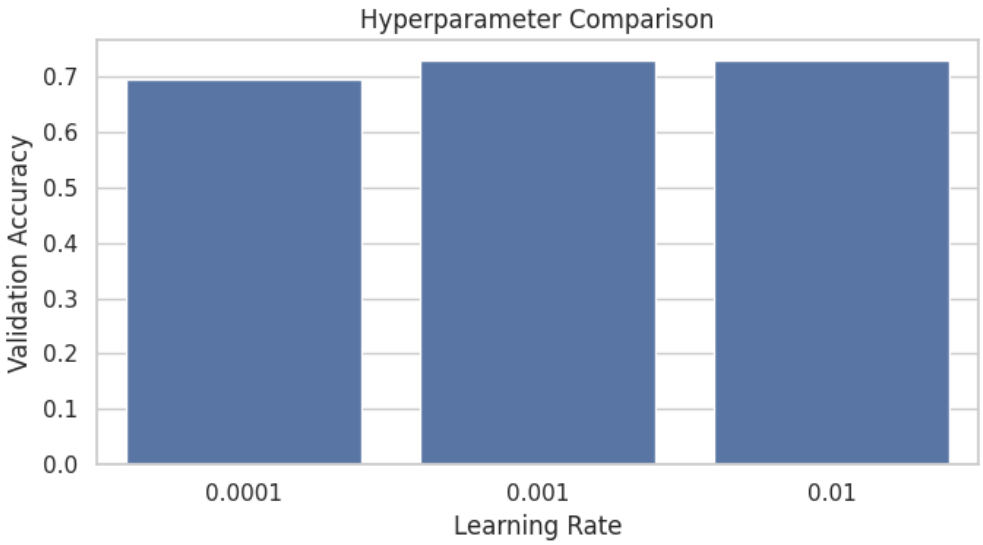


Figure 12. Validation accuracy for the tested learning rates.

Learning rate	Validation loss	Validation accuracy
0.0001	0.6020	0.6949
0.001	0.5615	0.7288
0.01	0.5621	0.7288

The learning rates 0.001 and 0.01 reached the same displayed validation accuracy (0.7288), while 0.001 had a slightly lower validation loss (0.5615 compared with 0.5621). Among the three tested values, 0.001 is therefore the preferred setting based on validation loss, although the experiment is limited and does not establish a globally optimal learning rate.

8. Discussion

The EDA stage confirms that the dataset is complete in this notebook run and highlights variation in grades and attendance. Absences are strongly right-skewed and include 15 potential IQR outliers. The MLP reaches 66.67% test accuracy, but its ROC-AUC of 0.6262 indicates only modest discrimination. Its higher recall for the higher-performance class and low recall for the lower-performance class show that the model is not equally reliable for both groups.

The model is a baseline experiment on a relatively small, single-subject dataset. The results should not be used to make consequential decisions about individual students. Additional validation, better class-sensitive evaluation, and careful consideration of data context would be needed before any practical use.

9. Limitations and Future Scope

- The dataset contains 395 records from the mathematics subject and may not represent other subjects, schools, or regions.
- The binary threshold at G3 = 10 simplifies a continuous grade scale and loses detail.
- The test set contains only 60 records, so individual errors can noticeably change the metrics.
- Lower-performance recall is 0.35, meaning many lower-performance students are missed.
- Permutation importance can vary with random sampling and correlated features and should not be interpreted causally.
- Future work can compare MLP with logistic regression, decision trees, random forests, and gradient boosting using the same split.
- Future experiments can evaluate class weights or threshold tuning, repeated stratified cross-validation, and confidence intervals.
- A future report can include additional subgroup checks and external validation before drawing practical conclusions.

10. Conclusion

This project completed an EDA workflow on the UCI Student Performance mathematics dataset and built a binary MLP classifier using demographic, family, school, lifestyle, and attendance features while excluding G1, G2, and G3 from the model inputs. The dataset contained 395 records, 33 columns, no missing values, and no duplicate rows in the executed notebook. Twelve visualization outputs were used to examine grades, study time, absences, correlations, training behaviour, test classification, feature importance, and hyperparameter comparison. The final MLP achieved 66.67% test accuracy, 0.7174 precision, 0.8250 recall, and 0.7674 F1-score for the higher-performance class, with ROC-AUC 0.6262. The results demonstrate the end-to-end workflow but also reveal that performance is limited, especially for the lower-performance class.

References

1. P. Cortez and A. Silva, “Using Data Mining to Predict Secondary School Student Performance,” Proceedings of the 5th Future Business Technology Conference, 2008.

2. UCI Machine Learning Repository, “Student Performance Data Set.” https://archive.ics.uci.edu/dataset/320/student+performance

3. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

4. M. Abadi et al., “TensorFlow: A System for Large-Scale Machine Learning,” 2016. https://www.tensorflow.org/

5. W. McKinney, “Data Structures for Statistical Computing in Python,” Proceedings of the Python in Science Conference, 2010.

Appendix A. Implementation Summary

The executed notebook uses Python, pandas, NumPy, Matplotlib, Seaborn, scikit-learn, and TensorFlow/Keras. It downloads the UCI mathematics CSV, inspects and cleans the data, performs EDA, constructs the binary target, splits the data with stratification, fits a preprocessing pipeline, trains an MLP, evaluates predictions, calculates permutation importance, compares learning rates, and saves the cleaned data and model artefacts.